Variability-Aware Analysis of Kbuild Makefiles

ANONYMOUS AUTHOR(S)

The Linux kernel is highly configurable, and its `kbuild` Makefiles decide how thousands of configuration options affect which source files are included in the kernel build. A developer who asks why a source file is not built, or which options would build it, needs its build condition, a logical formula over configuration options. Running Make reveals the files of one configuration only, and enumerating all configurations is intractable. Symbolic execution that forks a state at every conditional suffers from path explosion, and prior static analyses of Kbuild Makefiles represent a variable as a set of complete string values. We present KFOLD, a symbolic execution of `kbuild` Makefiles that maps each Make variable to a set of words, each guarded by a path condition over configuration options, in the style of a value summary. Appends add guarded words, an overwrite re-guards the words it replaces, and path merging joins the two arms of a conditional into one symbolic state, so the state grows with the number of words instead of the number of paths. The resulting build conditions are SMT formulae that a solver can check and satisfy.

We evaluate KFOLD on Linux 7.2.8 and four other `kbuild`-based systems by comparing its build conditions under concrete configurations with the source files built by fresh builds. On four Linux configurations, from `tinyconfig` to `allmodconfig`, the predicted files agree with the built files, with no false positive and no miss, from 512 to 29,556 files. KFOLD analyzes the 2,215 Makefile instances of the Linux x86 tree in 143.1 s. The build conditions explain the build status of all 400 sampled Linux files and, combined with a Kconfig model, produce configurations for 384 of the 402 commits of a Linux stable update that touch C or assembly files.

CCS Concepts: • **Software and its engineering → Software verification and validation**.

Additional Key Words and Phrases: Kbuild, Makefile, symbolic execution, variability, Kconfig

1 Introduction

placeholder: add later [?]

2 Background

3 Motivating Example

4 Approach

5 Implementation

6 Evaluation

6.1 Experimental Setup

6.2 RQ1

6.3 RQ2

7 Related Work

8 Conclusion

Data-Availability Statement

Link to repository removed for double-blind review.

AI Use

placeholder: add later (counts toward the 20-page limit)

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.